

Lecture 15: Routing — Link State and Dijkstra's Algorithm

EC 441 — Introduction to Computer Networking

Boston University, Spring 2026

Learning Objectives

By the end of this lecture, you should be able to:

1. Represent a network as a weighted graph, identifying nodes, edges, weights, and directionality
2. Explain how link-state routing works: each router learns the complete network topology through flooding
3. Describe the link-state advertisement (LSA) flooding process and why every router gets the same topology map
4. Execute Dijkstra's algorithm by hand on a 6–8 node network graph
5. Construct a shortest-path tree and derive a forwarding table from it
6. Explain how OSPF implements link-state routing in practice (areas, hello packets, LSA types)
7. Identify the oscillation problem in link-state routing with traffic-sensitive costs
8. Use Python's `networkx` library to build a graph, run Dijkstra, and visualize shortest-path trees

1 The Routing Problem

Lecture 13 established that every router has a **forwarding table**: given a destination prefix, the table says which output link to use. Lecture 14 explained what those prefixes mean (CIDR, subnetting). The question we have not yet answered: **how does the forwarding table get built?**

In a small network, a human could manually configure every entry. This is **static routing**, and it works for simple, rarely changing topologies. But it does not scale: a network with hundreds of routers and thousands of links has too many paths to configure by hand, and when a link fails, someone would have to manually reconfigure every affected router.

We need **dynamic routing**—algorithms that let routers automatically compute forwarding tables and adapt when the network changes. This is the **control plane**: the logic that builds the data plane's forwarding tables.

There are two major families of routing algorithms:

1. **Link-state** (this lecture)—every router learns the full topology, then independently computes shortest paths.
2. **Distance-vector** (Lecture 16)—routers learn only from their neighbors, iteratively converging on shortest paths.

A Useful Duality

These two families are, in a sense, duals of each other:

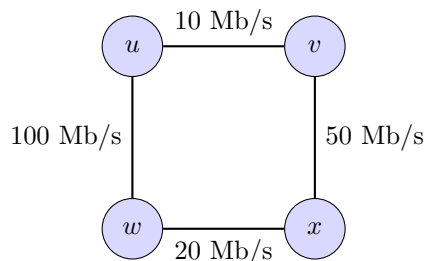
- **Link-state:** *local information distributed globally*—each router shares what it knows about its own links with every router in the network.
- **Distance-vector:** *global information distributed locally*—each router shares what it knows about all destinations with only its immediate neighbors.

Both model the network as a **graph**. Before we can talk about routing algorithms, we need the graph abstraction.

2 Networks as Graphs

A computer network is a collection of devices connected by links. This maps directly to a **graph** $G = (V, E)$:¹

- V —a set of **vertices** (also called **nodes**). In networking: routers.
- E —a set of **edges** (also called **links** or **arcs**). In networking: physical or logical connections between routers.



Edge Weights (Costs)

A **weighted graph** assigns a numerical value—a **weight** or **cost**—to each edge. The choice of weight determines what “shortest path” means:

Weight meaning	Lower is...	Used by
Hop count	Fewer routers traversed	RIP (all costs = 1)
Inverse bandwidth	Higher-capacity link	OSPF default: $10^8/B$
Propagation delay	Lower latency	Delay-sensitive routing
Administrative cost	Preferred by policy	Operator-configured
Current load	Less congested	Traffic engineering (problematic)

¹Wikipedia: Graph (discrete mathematics)—vertices, edges, directed/undirected, weighted graphs, paths, cycles.

The algorithm does not care what the weights represent—it just minimizes their sum along a path.

OSPF Default Cost

For a link with bandwidth B (in b/s), OSPF computes $\text{cost} = 10^8/B$. A 100 Mb/s Fast Ethernet link has cost 1; a 10 Mb/s Ethernet link has cost 10; a T1 (1.544 Mb/s) has cost 64. Modern implementations often raise the reference bandwidth to 10^9 or 10^{10} to differentiate Gigabit and 10-Gigabit links.

Physical interpretation: $10^8/B$ is the **transmission time** (in milliseconds) of 100 kb (12.5 kB) of data—approximately 8 normal-sized (1500-byte) packets. This connects OSPF cost directly to the transmission delay $d_{\text{trans}} = L/R$ from our delay model: a higher-bandwidth link has a lower transmission time and therefore a lower OSPF cost. Summing OSPF costs along a path is summing transmission delays—the algorithm minimizes the total time to push data across each link.

Directed vs. Undirected Edges

An **undirected edge** connects two nodes symmetrically—traversal works the same in both directions. Most full-duplex network links are naturally modeled this way.

A **directed edge** (or **arc**) goes one way. Directed edges model situations where the cost or capacity differs by direction—for example, a satellite uplink (slow) vs. downlink (fast). In this course, we primarily work with undirected graphs.

Paths and Path Cost

A **path** from vertex u to vertex v is a sequence of vertices connected by edges. The **cost of a path** is the sum of the edge weights along it:

$$\text{cost}(u \rightarrow w \rightarrow x \rightarrow v) = c(u, w) + c(w, x) + c(x, v)$$

A **shortest path** from u to v is a path with minimum total cost. There may be multiple shortest paths (ties), but the minimum cost is unique.

Key Definitions

- Two nodes are **adjacent** (or **neighbors**) if they share an edge.
- The **degree** of a node is the number of edges connected to it. A core ISP router might have degree 20+; an edge router serving one customer link might have degree 2.
- A **cycle** is a path that starts and ends at the same vertex. Real networks contain cycles for redundancy.
- A **tree** is a connected graph with no cycles. A tree with N vertices has exactly $N - 1$ edges.
- A **spanning tree** of G includes all vertices of G but only enough edges to keep it connected (no cycles).
- A **shortest-path tree (SPT)** rooted at vertex u is a spanning tree where the path from u to every other vertex is a shortest path in the original graph.

The Central Insight

A shortest-path tree is *exactly* what a router needs to build its forwarding table. Dijkstra's algorithm computes this tree.

Demo: traceroute

`traceroute` reveals the path packets take through the network—i.e., the path the routing algorithm has chosen. Each hop in the output is a vertex (router) in the network graph; each jump between hops is an edge.

```
$ traceroute google.com
1  192.168.1.1      1.2 ms  (home router)
2  10.0.0.1         5.4 ms  (ISP router)
3  72.14.215.85     8.8 ms  (Google edge)
4  142.250.80.46    9.1 ms  (destination)
```

`mtr` (“My TraceRoute”) combines `traceroute` with continuous `ping` for real-time per-hop statistics. Available on Linux natively; on macOS via Homebrew but requires `sudo` for raw socket access.

3 Link-State Routing

Link-state routing works in three phases:²

1. **Phase 1: Discover neighbors and measure link costs.** Each router sends **hello packets** on all its interfaces. When a neighbor responds, the router records: “I have a link to router *X* with cost *Y*.” This is a **Link-State Advertisement (LSA)**.
2. **Phase 2: Flood LSAs to all routers.** Each router broadcasts its LSA to every other router in the network. After flooding completes, every router has the same complete picture of the network topology—the full graph $G = (V, E)$ with all edge weights.
3. **Phase 3: Compute shortest paths.** Each router independently runs Dijkstra's algorithm on the shared topology to compute a shortest-path tree rooted at itself. From this tree, the router derives its forwarding table.

Why This Works

Because every router has the same topology database, and Dijkstra's algorithm is deterministic, every router computes **consistent** forwarding tables. There is no risk of routing loops (assuming the topology data is correct and up-to-date).

LSA Flooding Details

How does flooding ensure every router gets every LSA?

1. When router A generates an LSA, it sends it out all its interfaces.

²Wikipedia: Link-state routing protocol—flooding, LSAs, convergence, comparison with distance-vector.

2. When router B receives an LSA from A, it checks: have I seen this before (by sequence number)?
 - If **new**: store it, then forward out all interfaces *except* the one it arrived on.
 - If **already seen**: discard it. This prevents infinite flooding loops.
3. LSAs carry **sequence numbers** so routers can distinguish newer versions from stale ones.
4. LSAs have a **maximum age**—they expire and are re-flooded periodically (typically every 30 minutes in OSPF).

After a brief convergence period, every router has an identical copy of every other router's LSA—i.e., the complete topology.

4 Dijkstra's Algorithm

The Problem

Given a weighted graph $G = (V, E)$ and a source node s , find the shortest path from s to every other node in V .³

This was solved by Edsger Dijkstra in 1956 (published 1959).⁴ The algorithm is one of the most important in computer science and is used directly in OSPF, IS-IS, and many other systems.

The Algorithm

Dijkstra's algorithm maintains three data structures:

- $D(v)$ —the current best-known cost from s to v .
- $p(v)$ —the predecessor of v on the current best path (used to reconstruct the path afterward).
- N' —the set of nodes whose shortest path has been definitively determined (“finalized”).

Dijkstra's Algorithm — Pseudocode

```
Initialize:
  N' = {s}
  D(s) = 0
  For all neighbors v of s: D(v) = c(s,v), p(v) = s
  For all other nodes v:   D(v) = ∞

Loop (until N' contains all nodes):
  Find w ∉ N' with minimum D(w)
  Add w to N'
  For each neighbor v of w where v ∉ N':
    If D(w) + c(w,v) < D(v):      // relax
      D(v) = D(w) + c(w,v)
      p(v) = w
```

³Wikipedia: Dijkstra's algorithm—pseudocode, worked examples, complexity analysis, and history.

⁴Wikipedia: Edsger W. Dijkstra—biography of the algorithm's inventor.

Relaxation and Correctness

The inner operation—testing whether $D(u) + c(u, v) < D(v)$ and updating if so—is called **relaxation**. The name comes from constraint optimization: the distance estimate $D(v)$ is “too tight” (an overestimate) and we “relax” it down to a better value.

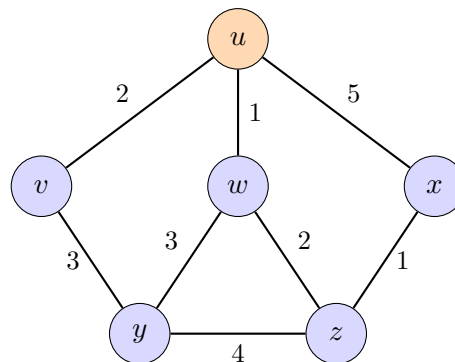
Why is the greedy choice correct? When we pick node w with minimum $D(w)$ from the un-finalized set Q , we can prove $D(w)$ is the true shortest-path cost:

- The finalized set N' and the remaining set Q form a **cut** in the graph.
- Any alternative path from s to w must cross this cut through some node $b \in Q$.
- But $D(b) \geq D(w)$ (we chose the minimum), and the remaining edges from b to w add non-negative weight.
- Therefore no alternative path can be shorter than $D(w)$.

This is why Dijkstra's algorithm **requires non-negative edge weights**. A negative edge after b could make an alternative path cheaper, breaking the greedy invariant. (Bellman-Ford, used in distance-vector routing, handles negative weights—but negative link costs are rare in practice.)

Worked Example

Consider the following 6-node network with source node u :



Edge costs: $c(u, v) = 2$, $c(u, w) = 1$, $c(u, x) = 5$, $c(v, y) = 3$, $c(w, y) = 3$, $c(w, z) = 2$, $c(x, z) = 1$, $c(y, z) = 4$.

Dijkstra Step-by-Step Trace

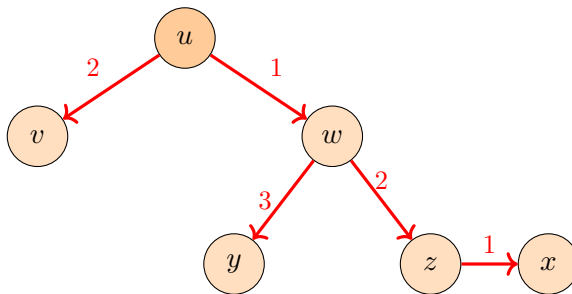
Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$	Select
Init	$\{u\}$	$2, u$	$1, u$	$5, u$	∞	∞	—
1	$\{u, w\}$	$2, u$	$1, u$	$5, u$	$4, w$	$3, w$	w
2	$\{u, w, v\}$	$2, u$	—	$5, u$	$4, w$	$3, w$	v
3	$\{u, w, v, z\}$	—	—	$5 \rightarrow 4, z$	$4, w$	$3, w$	z
4	$\{u, w, v, z, x\}$	—	—	$4, z$	$4, w$	—	x
5	$\{u, w, v, z, x, y\}$	—	—	—	$4, w$	—	y

Key observations:

- **Step 1:** w is finalized (cost 1). Its neighbors y and z are discovered.
- **Step 2:** v is finalized (cost 2). Checking y via v : $2 + 3 = 5 > 4$; no improvement.
- **Step 3:** z is finalized (cost 3). The cost to x *improves* from 5 to 4 via the path $u \rightarrow w \rightarrow z \rightarrow x$. The direct edge $u-x$ (cost 5) is not the shortest path!
- **Steps 4–5:** x and y are finalized (both cost 4).

The Shortest-Path Tree

From the predecessor pointers $p(v)$, we reconstruct the shortest-path tree rooted at u :



Destination	Shortest path from u	Cost
v	$u \rightarrow v$	2
w	$u \rightarrow w$	1
y	$u \rightarrow w \rightarrow y$	4
z	$u \rightarrow w \rightarrow z$	3
x	$u \rightarrow w \rightarrow z \rightarrow x$	4

From Shortest-Path Tree to Forwarding Table

The SPT tells router u : to reach any destination, which **first hop** should I use?

Destination	Next hop	Cost	Path
v	v	2	$u \rightarrow v$
w	w	1	$u \rightarrow w$
x	w	4	$u \rightarrow w \rightarrow z \rightarrow x$
y	w	4	$u \rightarrow w \rightarrow y$
z	w	3	$u \rightarrow w \rightarrow z$

Four of five destinations use next hop w . The router only needs to know the first hop, not the full path. In a real router, the “destination” column contains IP prefixes and “next hop” is a physical interface plus IP address.

Complexity

Dijkstra's algorithm runs in $O(|V|^2)$ with a simple array, or $O((|V| + |E|) \log |V|)$ with a binary-heap priority queue. For the sparse graphs typical of networks, the heap version is faster. For the small networks in this course, the difference is negligible.

5 Every Router Runs Dijkstra

In a link-state network with N routers, **every router** runs Dijkstra independently, using the same topology database. Each router computes a shortest-path tree rooted at **itself**, producing a different forwarding table.

When do routers recompute? Only when the topology changes:

1. A new LSA arrives indicating a link failure, recovery, or cost change.
2. An LSA ages out and is refreshed.
3. A new router joins the network.

In stable networks, Dijkstra runs infrequently—the forwarding table stays static for long periods. When a link fails, the affected routers generate new LSAs, flooding triggers recomputation at all routers, and the network **converges** on new shortest paths.

Convergence

Convergence is the process of all routers agreeing on a consistent set of forwarding tables after a topology change. Link-state protocols converge relatively quickly because flooding is fast and every router independently computes a correct result from the complete topology. Typical OSPF convergence time: 1–10 seconds after a link failure.

6 OSPF: Open Shortest Path First

OSPF is the most widely deployed link-state routing protocol.⁵ It implements exactly the three-phase process described above:

⁵Wikipedia: OSPF— protocol overview, areas, hello protocol, LSA types, and RFC references.

1. Routers discover neighbors via **hello packets** (default: every 10 s on broadcast networks).
2. Routers exchange link-state information via **LSAs**, building a **link-state database (LSDB)** at every router.
3. Each router runs **Dijkstra's algorithm** on the LSDB to compute shortest paths and populate its forwarding table.

OSPF runs directly over IP (protocol number 89—it is not a TCP or UDP application). It is defined in RFC 2328 (OSPFv2 for IPv4) and RFC 5340 (OSPFv3 for IPv6).

The Hello Protocol

OSPF routers send hello packets on each interface at regular intervals. Hello packets serve three purposes:

1. **Neighbor discovery**—finding adjacent routers.
2. **Bidirectional verification**—confirming that the neighbor sees you too. Adjacency is formed only after both routers list each other in their hello packets.
3. **Failure detection**—if hellos stop arriving for the **dead interval** (default 40 s), the neighbor is declared down and a new LSA is flooded.

OSPF Areas

In large networks, flooding LSAs everywhere and running Dijkstra on the full topology would be expensive. OSPF solves this with **areas**:

- The network is divided into areas, each identified by a 32-bit number.
- **Area 0** is the **backbone**—all other areas must connect to it.
- LSA flooding is contained within an area; routers only need the full topology of their own area.
- **Area Border Routers (ABRs)** connect areas and summarize routes between them.

For this course, we focus on the single-area case. The multi-area design matters for large deployments but the algorithmic principles are the same.

OSPF Packet Types

OSPF defines five packet types:

Type	Name	Purpose
1	Hello	Neighbor discovery and keep-alive
2	DB Description	Summarize LSDB during initial adjacency sync
3	LS Request	Request specific LSAs from a neighbor
4	LS Update	Carry one or more LSAs (the actual topology data)
5	LS Acknowledgment	Confirm receipt of LSAs (reliable flooding)

OSPF packets travel between routers, not end hosts. You will not see them in a Wireshark capture on a laptop; they are observable on router interfaces or in lab environments (GNS3, Mininet + FRRouting).⁶

7 The Oscillation Problem

What if link costs are based on current traffic load? This seems appealing—route traffic away from congested links automatically. But it creates an **oscillation** problem.

Traffic-Sensitive Cost Oscillation

Consider two paths between A and D . Initially both are idle (all costs 1), so the shortest path is the direct link $A \rightarrow D$. All traffic takes this path, making it congested (cost rises to 10). Now $A \rightarrow B \rightarrow C \rightarrow D$ becomes shorter (cost 3). All traffic shifts— $B \rightarrow C \rightarrow D$ becomes congested. The traffic shifts back. This cycle never stabilizes.

Solutions:

1. **Do not use load-sensitive costs.** OSPF and IS-IS use static costs based on bandwidth or administrative policy. This is standard practice today.
2. Dampen oscillations with exponentially weighted moving averages of load, using long time constants.
3. Randomize recomputation times so routers do not react to the same snapshot simultaneously.

Modern traffic engineering uses MPLS or segment routing for load-sensitive path selection, not dynamic OSPF cost changes.

8 Python Demos

Demo: networkx — Graph Construction and Dijkstra

Python's `networkx` library provides a clean API for building graphs and running shortest-path algorithms:

```
import networkx as nx

G = nx.Graph()
edges = [("u", "v", 2), ("u", "w", 1), ("u", "x", 5),
         ("v", "y", 3), ("w", "y", 3), ("w", "z", 2),
         ("x", "z", 1), ("y", "z", 4)]
for s, d, c in edges:
    G.add_edge(s, d, weight=c)

lengths, paths = nx.single_source_dijkstra(G, "u", weight="weight")
```

The exploration notebook (`lecture_15_exploration.py`) provides interactive visualization: side-by-side full graph vs. SPT, forwarding table derivation, and link-failure simulation.

⁶Wikipedia: IS-IS—the other major link-state protocol, used by large ISPs. IS-IS runs over Layer 2 rather than over IP.

Demo: From-Scratch Dijkstra (demo_dijkstra_l15.py)

A standalone implementation matching the pseudocode from this lecture:

```
def relax(u, v, G):
    if G.V[v].d > G.V[u].d + G.w[(u, v)]:
        G.V[v].d = G.V[u].d + G.w[(u, v)]
        G.V[v].pi = u

def dijkstra(G, source):
    G.initialize_source(source)
    Q = set(G.V.keys())
    while Q:
        u = min(Q, key=lambda x: G.V[x].d)
        Q.remove(u)
        for v in G.Adj[u]:
            if v in Q:
                relax(u, v, G)
```

The full script prints step-by-step trace tables matching the slides, reconstructs paths from predecessor pointers, derives a forwarding table, and simulates link-failure rerouting.

Discussion Questions

1. In the worked example, step 3 improved $D(x)$ from 5 to 4 by going through z instead of directly from u . Can a shortest path to a node be discovered “late” in Dijkstra’s algorithm? Why does the algorithm still guarantee correctness?
2. If all edge weights in a graph are equal (say, all 1), what does Dijkstra’s algorithm reduce to? What simpler algorithm would you use instead?
3. OSPF floods LSAs to all routers. In a network with 100 routers and 300 links, how many LSAs does each router store? What is the total number of LSA messages transmitted during a single flooding event triggered by one link failure?
4. A router has two equal-cost paths to a destination (cost 10 each). How should it forward traffic? Does OSPF handle this case? (Look up: ECMP—Equal-Cost Multi-Path.)
5. Why does Dijkstra’s algorithm fail with negative edge weights? Construct a small example (3–4 nodes) where a negative weight causes the algorithm to produce an incorrect shortest path.
6. Link-state routing requires every router to have the complete topology. Distance-vector routing requires only neighbor information. What are the tradeoffs in terms of (a) memory, (b) bandwidth, and (c) convergence speed?
7. An ISP operator wants to force traffic away from an expensive satellite link and toward cheaper terrestrial fiber. How would they accomplish this using OSPF link costs?
8. The oscillation problem shows that load-sensitive routing can be unstable. Yet modern CDNs and cloud providers do route traffic based on load. How do they avoid the oscillation problem?

Further Reading

The following Wikipedia articles are the primary student-facing references for this lecture.

- Wikipedia: Graph (discrete mathematics)—vertices, edges, directed/undirected, weighted graphs, paths, and cycles.
- Wikipedia: Dijkstra's algorithm—algorithm description, pseudocode, complexity analysis, and history.
- Wikipedia: Shortest-path tree—definition, relation to spanning trees, construction via shortest-path algorithms.
- Wikipedia: Link-state routing protocol—flooding, LSAs, link-state database, convergence, and comparison with distance-vector.
- Wikipedia: OSPF— protocol overview, areas, hello protocol, LSA types, RFC 2328.
- Wikipedia: Edsger W. Dijkstra—biography of the algorithm's inventor.
- Wikipedia: Spanning tree— spanning trees, minimum spanning trees, and their relation to network graphs.
- Wikipedia: Routing—general overview of routing concepts, static vs. dynamic routing, and algorithm families.
- Wikipedia: Interior gateway protocol—IGP classification; OSPF vs. IS-IS vs. RIP.
- Wikipedia: IS-IS—the other major link-state protocol, used by large ISPs; runs over Layer 2.
- Wikipedia: Convergence (routing)—convergence time, transient loops, and factors affecting convergence speed.
- Wikipedia: Equal-cost multi-path routing—ECMP, load balancing across multiple equal-cost shortest paths.

Prepared for EC 441, Boston University, Spring 2026.